Guia técnico — Parte do Aluno 1

Programação Distribuída | visão de implementação e integração | revisão

de 20/10/2026

GUIA EXPLICATIVO

Fluxos, módulos, funções, decisões, evidências e pendências

Leitura baseada no código e na documentação do repositório

Revisão: 20/10/2026

Uso didático: implementação atual separada de requisitos e trabalho futuro

1. Objetivo e leitura do estado

Este guia acompanha o código e os registros do projeto. Ele explica os fluxos que estão implementados, as responsabilidades atribuídas ao Aluno 1, as escolhas feitas e as diferenças entre o comportamento atual e o objetivo geral do trabalho. “Esperado” descreve o contrato do checkpoint; “implementado” descreve apenas o que o código e as evidências disponíveis sustentam.

O Aluno 1 concentra a comunicação e o produto de arquivos: framing TCP, armazenamento dos documentos nos Peers, transferência de chunks, metadados publicados no Super Peer e, no C3, o serviço de membership, heartbeat e disseminação Gossip. A topologia Chord e a identidade NodeID são responsabilidades do Aluno 2, embora os módulos dos dois alunos componham os mesmos executáveis.

2. Arquitetura em uma página

```
                    controle e catálogo
CLI local --IPC--> Peer ativo --TCP JOIN/HEARTBEAT/ANNOUNCE--> Super Peer
                       |                                          |
                       | TCP STORE/GET de chunks                  | índice local
                       v                                          | metadados e
                 outros Peers <------------ LOOKUP -------------+ localizações

  Upload:   arquivo -> ObjectID -> chunks -> compressão -> Peer -> ANNOUNCE no Super Peer
  Download: LOOKUP -> localizações -> chunks direto dos Peers -> validação -> arquivo final
```

O Super Peer guarda metadados e localizações; ele não armazena os bytes do PDF. Cada Peer conserva seus chunks e manifests. A CLI conversa com o Peer ativo por canal Unix privado; esse canal escolhe um processo local que já ingressou na rede, não cria um Peer anônimo.

Os executáveis padrão são bin/superpeer e bin/peer. Os nomes bin/node e bin/client são links simbólicos para manter compatibilidade com roteiros antigos. Os dois processos têm main próprios. concurrent_server.c compartilha o ciclo de atendimento por conexão.

3. Checkpoint 1: transporte e protocolo

Caminho de uma mensagem

- A aplicação preenche tipo, origem, destino, TransactionID e payload.
- protocol_send_message valida os limites, serializa o header e calcula o CRC32.
- network_send_all repete send até enviar todos os bytes, inclusive quando o sistema aceita somente uma parte.
- O receptor usa network_recv_exact para obter o header fixo e depois exatamente o payload indicado pelo tamanho.
- protocol_deserialize_header e protocol_validate_header verificam a estrutura; protocol_receive_message confere o CRC antes de entregar a mensagem ao handler.

O header tem 98 bytes, os inteiros wire usam big-endian, o TransactionID tem 16 bytes e o payload máximo é 5 MiB. O CRC32 cobre header e payload. A conexão TCP é um fluxo de bytes, não preserva fronteiras de mensagens; por isso o tamanho do payload faz parte do framing.

Módulos e funções importantes

- network.c: network_create_server, network_connect, network_send_all, network_recv_exact e network_shutdown encapsulam sockets POSIX e fechamento previsível.

- protocol.c: protocol_serialize_header, protocol_deserialize_header, protocol_validate_header, protocol_send_message e protocol_receive_message definem o formato wire comum.

- concurrent_server.c: aceita conexões e administra workers para que um cliente lento não bloqueie todos os outros.

- rpc.c: implementa chamadas request/response com prazo, correlação, endereço e identidade do destino.

- peer.c: inicia o servidor do Peer, processa transferência de chunks, JOIN e controles relacionados.

- superpeer.c: recebe JOIN, STORE/ANNOUNCE e LOOKUP, além de integrar os serviços C3/C4.

As mensagens usam serialização explícita. O programa não envia struct C diretamente pela rede: padding, alinhamento e ordem de bytes variam entre compiladores e arquiteturas.

4. Checkpoint 2: documentos e chunks

Upload esperado e implementado

```
arquivo PDF
   | validação da extensão e leitura incremental
   v
ObjectID = SHA-256(arquivo inteiro)
   | divisão em chunks de 4 MiB
   +- cada chunk: SHA-256 dos bytes originais + compressão LZ4
   v
workers enviam STORE/CHUNK a um Peer
   | Peer valida, grava em pending e confirma o documento
   v
COMMIT recompõe e verifica o arquivo -> publicação em objects
   |
   +- ANNOUNCE informa ao Super Peer metadados e localizações
```

- content_validate_pdf aceita extensão .pdf sem exigir uma assinatura interna, permitindo também fixtures sintéticas.

- object_id_file calcula o ObjectID do arquivo inteiro; os chunks têm 4 MiB (4 MiB = 4.194.304 bytes).

- file_client_upload divide os trabalhos por índice e usa pread, para workers independentes não compartilharem posição de arquivo.

- Cada chunk recebe SHA-256 dos dados não comprimidos e LZ4 individual. Workers abrem conexões próprias e enviam os dados e descritores.

- storage_begin cria ou retoma estado pendente. storage_put_chunk valida índice, offset, tamanho e hash antes de persistir o chunk.

- storage_commit remonta em ordem, confere tamanho e ObjectID e só então publica o manifesto final.

- announce_document codifica o catálogo e pede ao Super Peer que registre metadados e localizações. O cliente só confirma sucesso depois do ACK.

O processo do Peer armazena os bytes. directory.c adapta mensagens de rede para a API de metadados do Super Peer. O índice não recebe os próprios arquivos.

Download esperado e implementado

- file_client_download envia LOOKUP por nome ou ObjectID.
- O Super Peer retorna o registro do documento e as localizações disponíveis para cada chunk.
- O destino temporário <destino>.part é criado sem sobrescrever um arquivo existente.
- Workers pedem chunks diretamente aos Peers. Se uma localização falha, tentam a próxima localização anunciada.
- O Peer valida framing/CRC, descritor, descompressão LZ4 e SHA-256 por chunk; pwrite coloca cada bloco no offset esperado.
- Ao final, o cliente sincroniza e calcula o SHA-256 do arquivo recomposto. Só publica o destino final se ele coincide com o ObjectID.

Persistência e funções de armazenamento

storage.c usa .peer_storage/<porta>/pending para documentos incompletos e objects para documentos confirmados. Os caminhos são formados pelo ObjectID, nunca por nomes recebidos da rede. Manifests binários versionados guardam descritores, estado e dono. Escritas usam arquivos temporários, fsync e rename para evitar publicar conteúdo pela metade. Na inicialização, manifests são revalidados; documentos finais íntegros são anunciados novamente após o JOIN.

Funções para localizar no fluxo: storage_create, storage_begin, storage_put_chunk, storage_commit, storage_find, storage_read_chunk, storage_list e storage_descriptors. No cliente: file_client_upload, file_client_download, lookup_document, upload_worker, download_worker e execute_workers.

5. Concorrência e escolhas técnicas do C2

- O limite é 5 MiB para permitir que a representação comprimida de um chunk de 4 MiB caiba em uma mensagem.
- LZ4 é por chunk: o Peer valida e serve blocos individualmente, e uma falha não exige retransmitir o documento inteiro.
- Pools POSIX processam chunks em paralelo. O padrão limita workers ao mínimo entre CPUs, chunks e oito; PEER_TRANSFER_THREADS permite configurar de 1 a 32.
- Cada worker usa uma conexão TCP própria. Isso separa o estado do framing e permite que um chunk tente outra localização sem coordenar o socket compartilhado.
- Uma fila de trabalhos e o primeiro erro são protegidos por mutex. Workers não avançam sobre o mesmo chunk.
- Um arquivo parcial nunca aparece como destino final; o nome temporário e o rename protegem a publicação.
- SHA-256 é fornecido por libcrypto, CRC32 por zlib e LZ4 pela biblioteca externa liblz4.so.1.

São escolhas de confiabilidade e simplicidade. O benchmark existe, mas os resultados históricos não demonstram por si só a meta global de throughput de 500 MB/s.

6. Checkpoint 3: membership, heartbeat e Gossip

No C3, a parte do Aluno 1 mantém a visão de membros e detecta falhas. O Aluno 2 fornece identidade e Chord; a integração do detector com o overlay invalida vizinhos quando há falha.

```
heartbeat direto válido --------------------------> ALIVE
ausência de evidência direta ----------------------> SUSPECT
tempo excedido + duas sondagens negativas ---------> FAILED
tempo excedido após FAILED ------------------------> REMOVED (tombstone)
```

Os padrões documentados são heartbeat de 5 s, suspeita após 15 s, falha após 20 s e remoção após 30 s. Duas sondagens negativas precisam ocorrer depois da suspeita, com pelo menos 1 s entre elas. São limiares configuráveis, não prova matemática de que o processo morreu: uma partição ou atraso de rede também pode causar suspeita.

Funções e módulos C3

- membership.c: tabela sob mutex, registro/consulta, snapshots, transições, merge e persistência. APIs superpeer_ permanecem estáveis para não quebrar consumidores C1/C2.
- membership_heartbeat, membership_tick, membership_probe e membership_merge: aplicam evidência direta, prazos, sondagens e regras determinísticas de merge.
- heartbeat.c: heartbeat_create, heartbeat_start, heartbeat_handle, heartbeat_register, heartbeat_discover e heartbeat_peer_target coordenam o serviço e o reconectar do Peer.
- gossip.c: gossip_encode_record e gossip_decode_record codificam campos wire sem expor ponteiros, padding ou relógios locais.
- peer.c: emite heartbeats, responde a controles, refaz JOIN no endpoint configurado e anuncia manifests depois de se recuperar.
- superpeer.c: trata heartbeat/Gossip, aplica as consequências locais da falha e integra o detector ao Chord.
- concurrent_server.c: espera atendimentos ativos antes de destruir o contexto compartilhado.

O registro Gossip tem 186 bytes; HEARTBEAT carrega um registro com versão, totalizando 187 bytes. GOSSIP leva o estado do emissor e até 64 registros. Incarnação, sequência, estado, observador e versão resolvem mensagens repetidas ou atrasadas. A cópia ALIVE recebida por Gossip não substitui heartbeat direto para renovar disponibilidade.

membership.bin mantém o ledger persistente; node.incarnation distingue reinícios da mesma identidade e node.uuid mantém o NodeID estável. O disco não guarda bytes de PDF. O tombstone REMOVED impede que mensagem antiga ressuscite um nó removido.

Como deveria funcionar a recuperação

Um Peer reiniciado conserva seus chunks e manifest, incrementa incarnação, refaz JOIN e anuncia documentos concluídos. O Super Peer deve reconhecer a nova instância antes de aceitar mensagens antigas. Se o membro é outro Super Peer, a integração chama chord_forget ao marcar FAILED/REMOVED; a parte do Aluno 2 repara a topologia e só libera o NodeID de novo depois de uma confirmação direta da instância atual.

7. O que está feito e o que permanece

Evidência registrada para C1/C2

Os documentos do projeto registram C1 10/10, roteiro Peer anterior 14/14, C2 próprio 20/20, integração C2 com 46 verificações, comparação byte a byte, transferências multichunk, fallback de localização, reinicialização, identidade estável, anúncio repetido idempotente e rejeições. Também registram build C2x com -Werror e execuções isoladas ASan/UBSan e ThreadSanitizer. Esses são resultados datados no repositório; não significam que todo teste foi repetido nesta sessão.

Para o C3 integrado, make test-c3 reúne APIs locais e TCP. O relatório registra testes de três e cinco Super Peers, queda/reparo de sucessor, download íntegro durante reparo e volta do Super Peer com a mesma identidade e incarnação maior. Teste da API local e integração real por TCP são evidências distintas.

Pendências e fronteiras

- O índice de documentos permanece local ao Super Peer; Gossip de membership e Chord não replicam nem migram metadados.
- Uma consulta Chord localiza um responsável por chave, mas directory_lookup ainda não encaminha a consulta distribuída a qualquer Super Peer.
- Consistência forte entre catálogos, disponibilidade de 99,5%, lookup médio O(log N) e throughput acima da meta não estão demonstrados globalmente.
- Checkpoints posteriores do Aluno 1: cache LFU, cache hit/miss e eviction; integração final com réplica/transferência. Não declarar esses itens como prontos com base em chunks persistentes ou Gossip.
- A falha de rede pode ser confundida com falha de processo; o detector reduz falsos positivos com confirmação, mas não elimina a ambiguidade de uma partição.

8. Como demonstrar a parte do Aluno 1

Abra terminais separados na raiz do projeto:

```
make
./bin/superpeer --port 55101 --name superpeer
./bin/peer serve 55102 127.0.0.1 55101
./bin/peer serve 55103 127.0.0.1 55101
./bin/peer upload trabalho_2026_SD.pdf 127.0.0.1 55102
./bin/peer download trabalho_2026_SD.pdf copia.pdf 127.0.0.1 55101
```

Para integração C3, iniciar Super Peers adicionais no overlay é uma tarefa compartilhada: o serviço do Aluno 1 detecta estados; Chord do Aluno 2 mantém a topologia. Use diretórios de armazenamento exclusivos por processo e preserve manifests/UUID/incarnação ao demonstrar reinício.

9. Explicação curta para apresentação

"O Peer mantém os bytes dos arquivos e o Super Peer mantém o catálogo e as localizações. No upload, o cliente calcula o SHA-256 do arquivo, divide em chunks, comprime cada um e envia em paralelo. O Peer valida e persiste os blocos antes de confirmar o documento; só então anuncia os metadados. No download, o Super Peer indica quais Peers possuem os chunks, e o cliente busca diretamente deles, valida cada bloco e o arquivo completo e publica o destino sem sobrescrever arquivos. No C3, heartbeats diretos e sondagens sustentam os estados de falha, enquanto Gossip dissemina eventos. Essas evidências de membership não significam consenso nem replicação do catálogo."

10. Referências internas

Fontes principais: requisitos_aluno_1.md, explicacao_codigo_aluno_1.md, checkpoint_3_aluno_1.md, integracao_checkpoint_3.md, protocolo_checkpoint_2.md, refatoracao_checkpoint_2.md, headers e implementações dos módulos citados. Os requisitos históricos ficam separados do estado atual pelos relatórios datados.